

Software Testing

CONTENT

- Introduction
- What is Software Testing ?
- Objectives of Testing
- Goals of Testing
- Testing Methodologies
- Software Testing lifecycle – Phases
- Testing Levels
- Types of Performance Testing
- Conclusion

INTRODUCTION

- Testing is a process used to help identify the correctness, completeness and quality of developed computer software. With that in mind, testing can never completely establish the correctness of computer software.
- There are many approaches to software testing from using alm tools to automated testing, but effective testing of complex products is essentially a process of investigation, not merely a matter of creating and following rote procedure.

WHAT IS SOFTWARE TESTING?

Executing software in a simulated or real environment, using inputs selected randomly.

OBJECTIVES OF TESTING

- Uncover as many as errors (or bugs) as possible in a given product.
- Demonstrate a given software product matching its requirement specifications.
- Validate the quality of a software testing using the minimum cost and efforts.
- Generate high quality test cases, perform effective tests, and issue correct and helpful problem reports.

ERROR, BUG, FAULT & FAILURE

Error : It is a human action that produces the incorrect result that produces a fault.

Bug : The presence of error at the time of execution of the software.

Fault : State of software caused by an error.

Failure : Deviation of the software from its expected result. It is an event.

GOALS OF TESTING

- ⦿ Detect faults
- ⦿ Establish confidence in software
- ⦿ Evaluate properties of software
 - Reliability
 - Performance
 - Memory Usage
 - Security
 - Usability

TESTING METHODOLOGIES

- ◉ White box testing
- ◉ Black-box testing

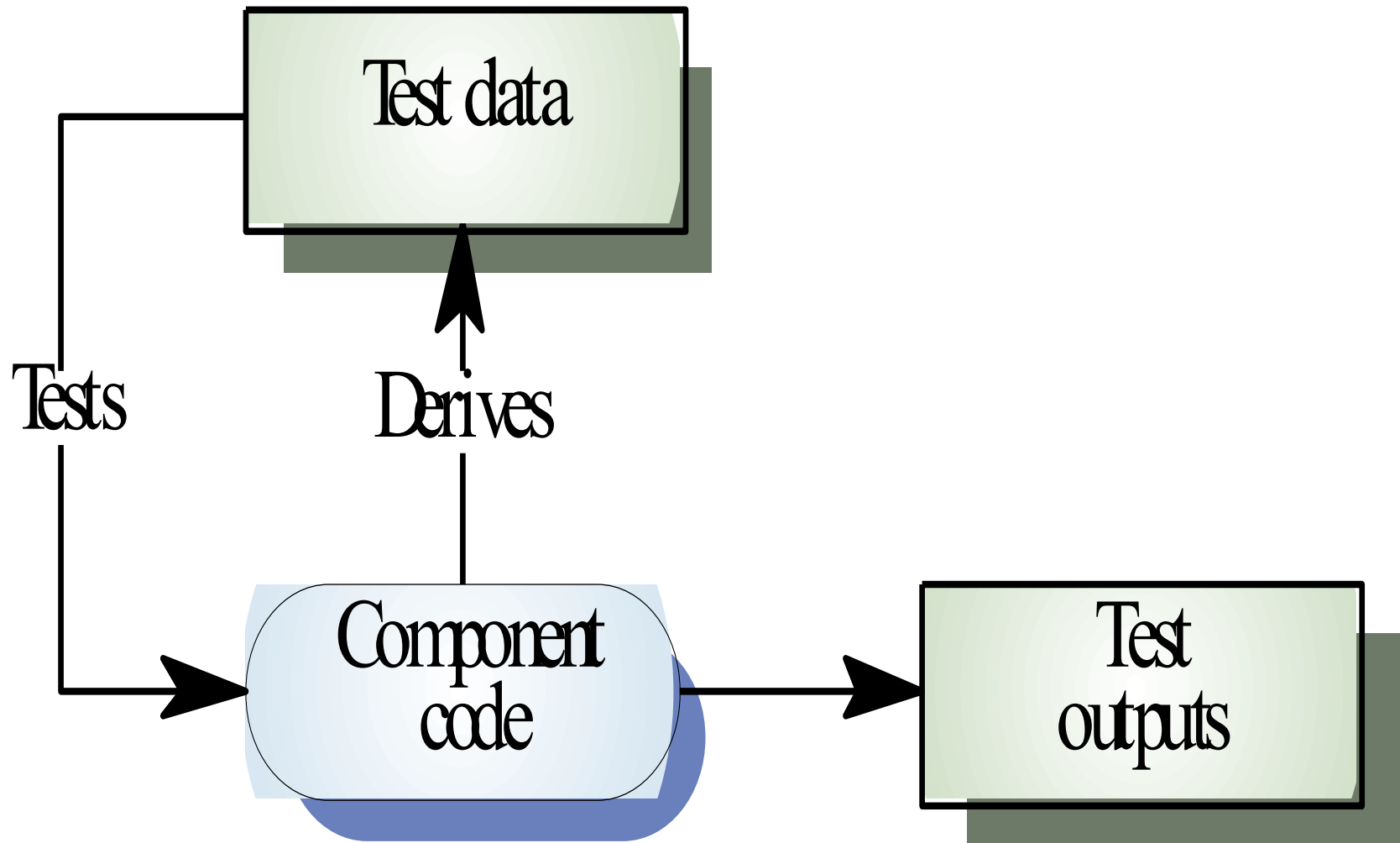
◎ **White box testing**

- Knowledge of the internal program design and code required.
- Tests are based on coverage of code statements, branches, paths, conditions.

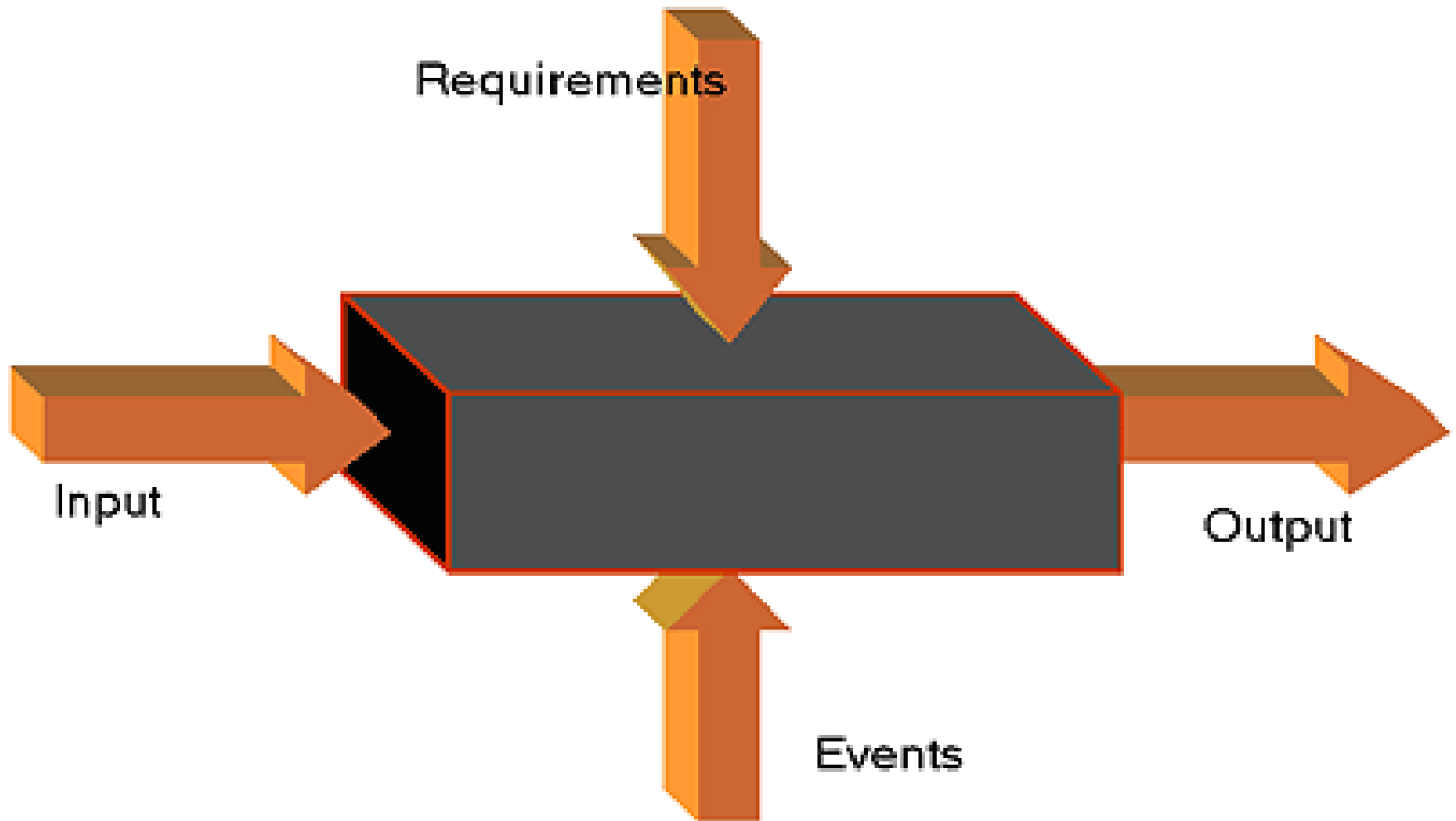
◎ **Black box testing**

- No knowledge of internal program design or code required.
- Tests are based on requirements and functionality.

WHITE-BOX TESTING



BLACK-BOX TESTING



SOFTWARE TESTING LIFECYCLE

- PHASES

- Requirements study
- Test Case Design and Development
- Test Execution
- Test Closure
- Test Process Analysis

REQUIREMENTS STUDY

- ⦿ Testing Cycle starts with the study of client's requirements.
- ⦿ Understanding of the requirements is very essential for testing the product.

ANALYSIS & PLANNING

- Test objective and coverage
- Overall schedule
- Standards and Methodologies
- Resources required, including necessary training
- Roles and responsibilities of the team members
- Tools used

Test Case Design and Development

- Component Identification
- Test Specification Design
- Test Specification Review

Test Execution

- Code Review
- Test execution and evaluation
- Performance and simulation

Test Closure

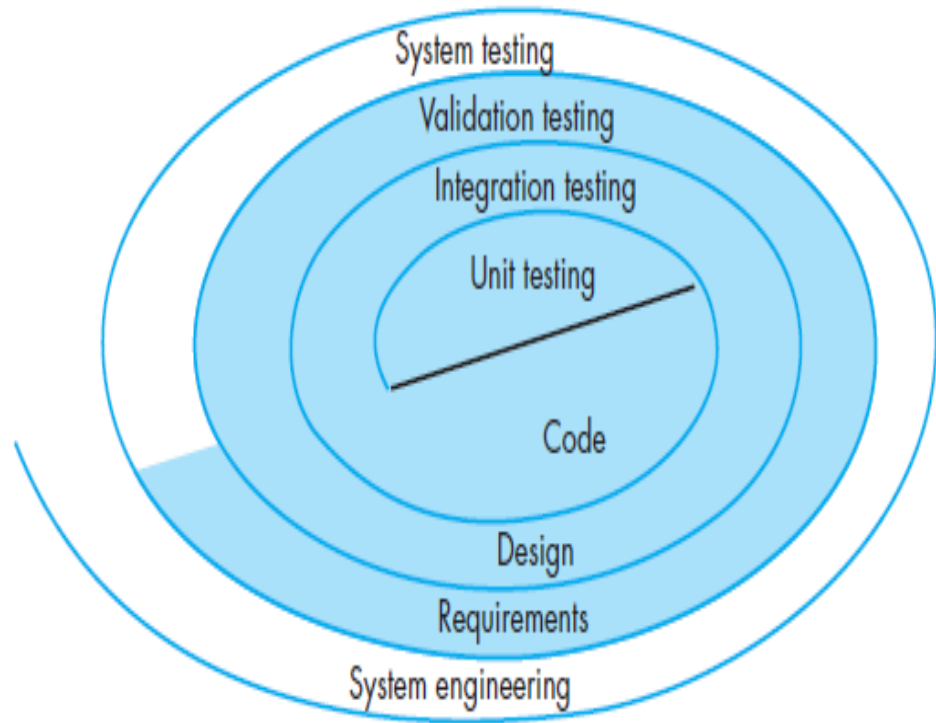
- Test summary report
- Project De-brief
- Project Documentation

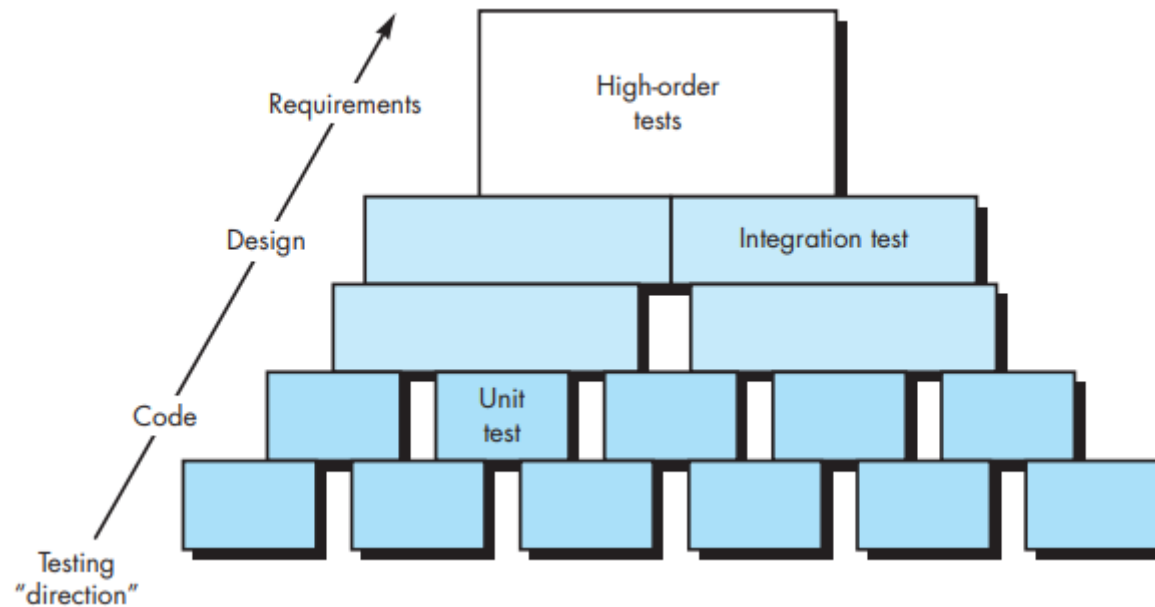
Test Process Analysis

Analysis done on the reports and improving the application's performance by implementing new technology and additional features.

TESTING LEVELS

- Unit testing
- Integration testing
- System testing

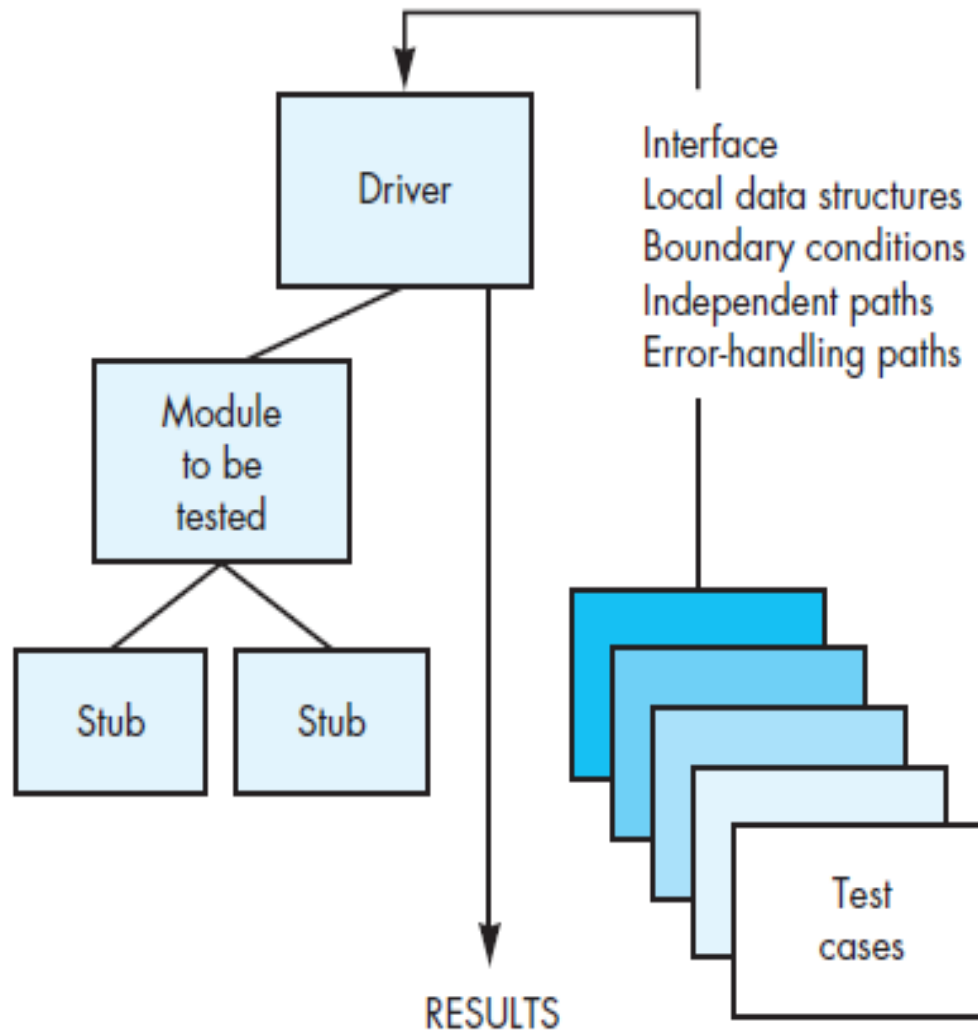




Software testing steps

UNIT TESTING

- Tests each module individually.
- Follows a white box testing (Logic of the program).
- Done by developers.



INTEGRATION TESTING

- Once all the modules have been unit tested, integration testing is performed.
- It is systematic testing.
- Produce tests to identify errors associated with interfacing.

Types:

- Big Bang Integration testing
- Top Down Integration testing
- Bottom Up Integration testing
- Mixed Integration testing

BIG-BANG INTEGRATION TESTING

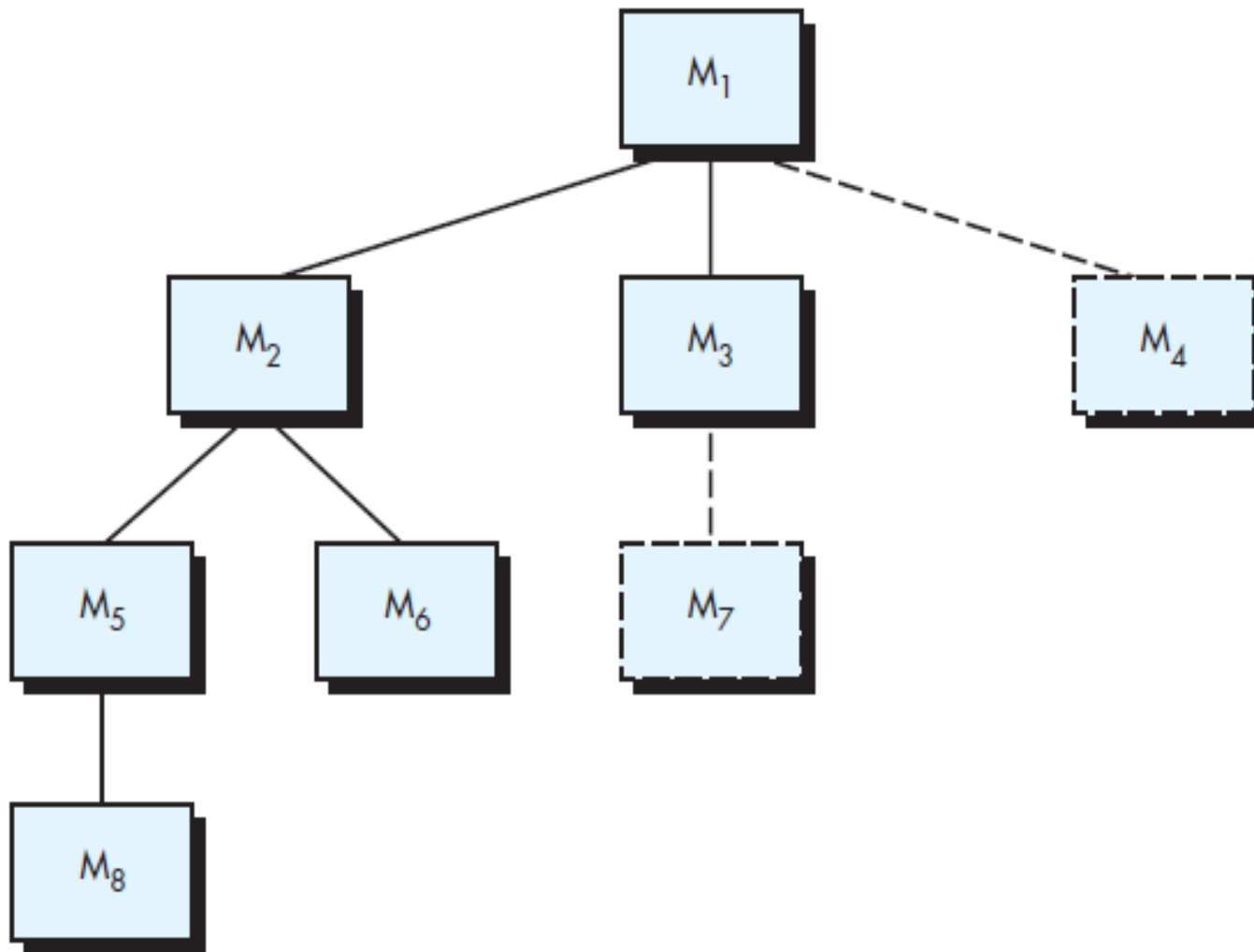
- ◉ It is the simplest integration testing approach, where all the modules making up a system are integrated in a single step.
- ◉ In simple words, all the modules of the system are simply put together and tested. This technique is practicable only for very small systems.
- ◉ The main problem with this approach is that once an error is found during the integration testing, it is very difficult to localize the error as the error may potentially belong to any of the modules being integrated.
- ◉ Therefore, debugging errors reported during big bang integration testing are very expensive to fix.

BOTTOM-UP INTEGRATION TESTING

- ◉ In bottom-up testing, each subsystem is tested separately and then the full system is tested.
- ◉ A subsystem might consist of many modules which communicate among each other through well-defined interfaces.
- ◉ The primary purpose of testing each subsystem is to test the interfaces among various modules making up the subsystem.
- ◉ Both control and data interfaces are tested.
- ◉ The test cases must be carefully chosen to exercise the interfaces in all possible manners
- ◉ Large software systems normally require several levels of subsystem testing; lower-level subsystems are successively combined to form higher-level subsystems.
- ◉ A principal advantage of bottom-up integration testing is that several disjoint subsystems can be tested simultaneously.
- ◉ In a pure bottom-up testing no stubs are required, only test-drivers are required.
- ◉ A disadvantage of bottom-up testing is the complexity that occurs when the system is made up of a large number of small subsystems.
- ◉ The extreme case corresponds to the big-bang approach.

TOP-DOWN INTEGRATION TESTING

- ⦿ Top-down integration testing starts with the main routine and one or two subordinate routines in the system.
- ⦿ After the top-level ‘skeleton’ has been tested, the immediately subroutines of the ‘skeleton’ are combined with it and tested.
- ⦿ Top-down integration testing approach requires the use of program stubs to simulate the effect of lower-level routines that are called by the routines under test.
- ⦿ A pure top-down integration does not require any driver routines.
- ⦿ A disadvantage of the top-down integration testing approach is that in the absence of lower-level routines, many times it may become difficult to exercise the top-level routines in the desired manner since the lower-level routines perform several low-level functions such as I/O.



Top-down integration

MIXED INTEGRATION TESTING

- ⦿ A mixed (also called sandwiched) integration testing follows a combination of top-down and bottom-up testing approaches.
- ⦿ In top-down approach, testing can start only after the top-level modules have been coded and unit tested.
- ⦿ Similarly, bottom-up testing can start only after the bottom level modules are ready.
- ⦿ The mixed approach overcomes this shortcoming of the top-down and bottom-up approaches.
- ⦿ In the mixed testing approaches, testing can start as and when modules become available.
- ⦿ Therefore, this is one of the most commonly used integration testing approaches

PHASED VS. INCREMENTAL TESTING

- ◎ The different integration testing strategies are either phased or incremental. A comparison of these two strategies is as follows:
 - In incremental integration testing, only one new module is added to the partial system each time.
 - In phased integration, a group of related modules are added to the partial system each time.

APPROACHES TO INTEGRATION TESTING

- Top-down testing

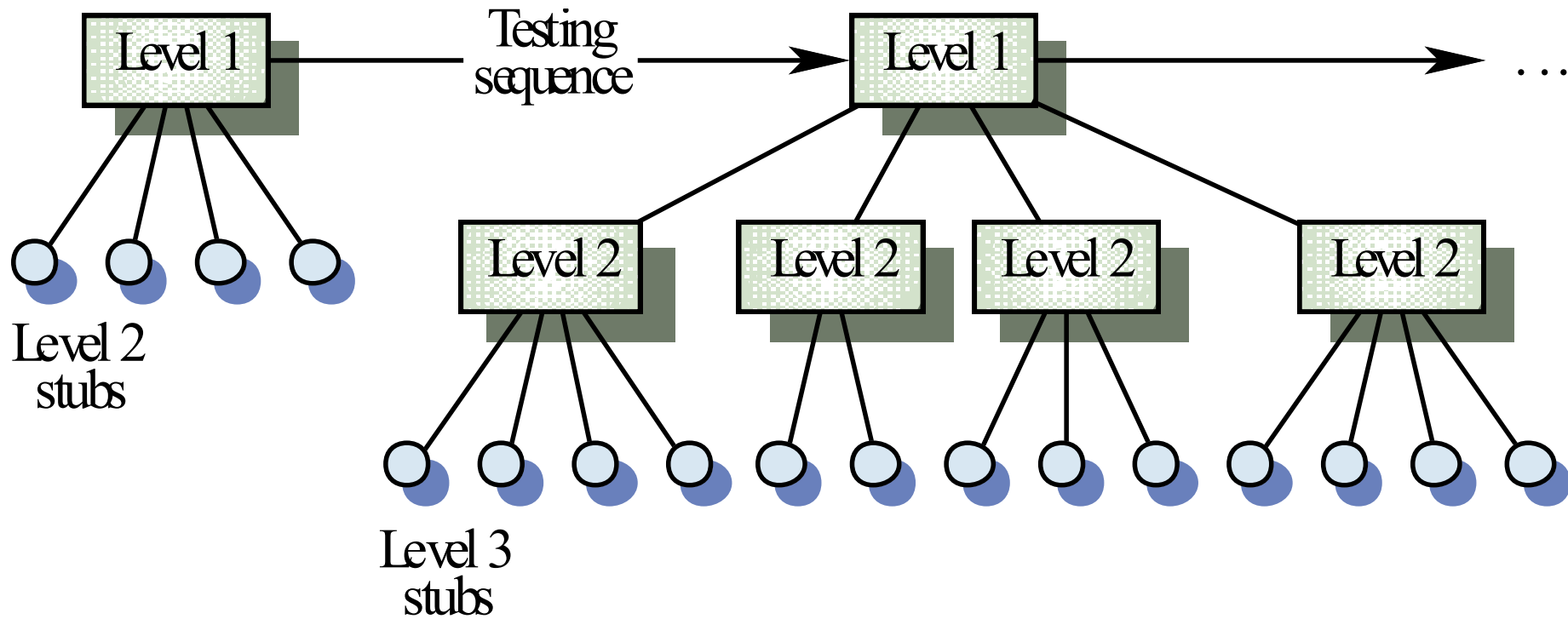
- Start with high-level system and integrate from the top-down replacing individual components by stubs where appropriate

- Bottom-up testing

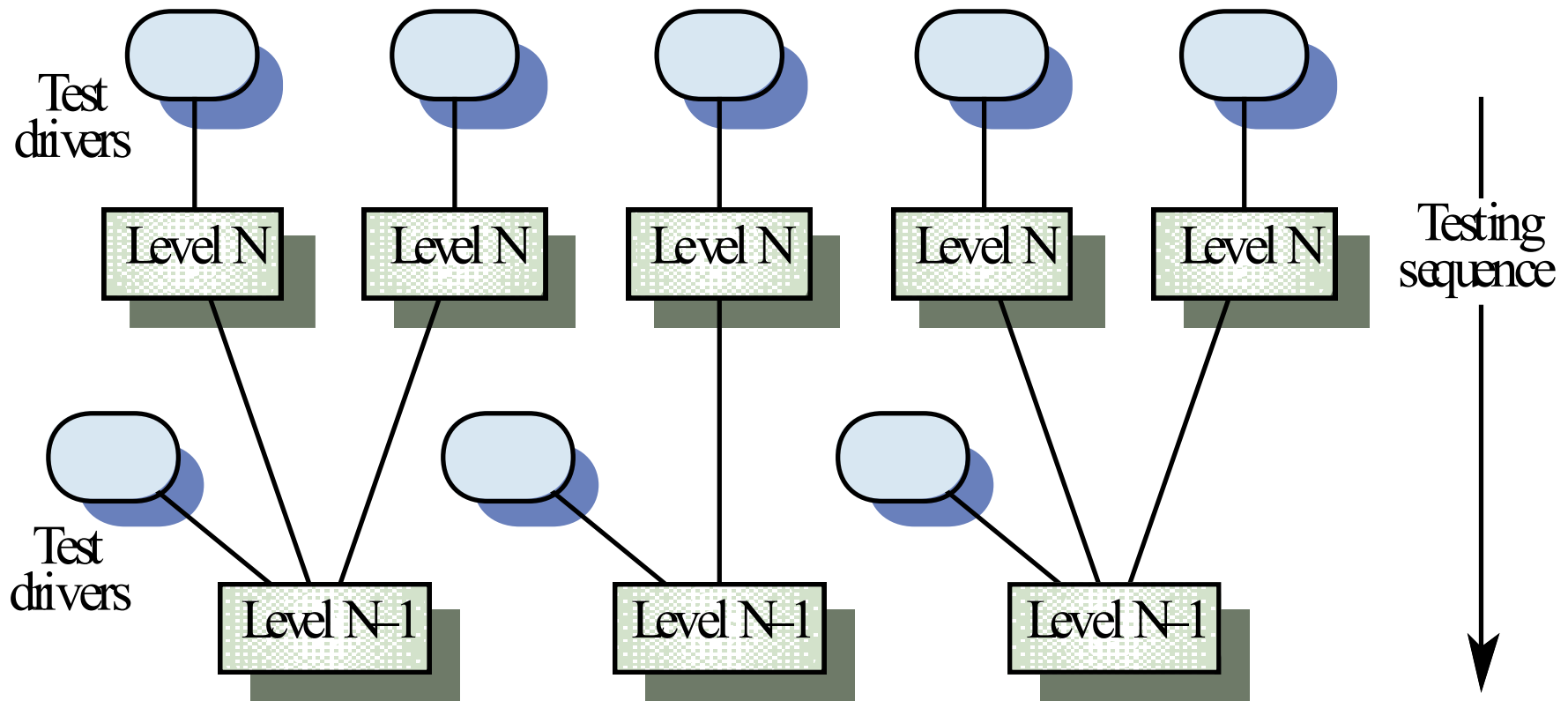
- Integrate individual components in levels until the complete system is created

- In practice, most integration involves a combination of these strategies

TOP-DOWN TESTING



BOTTOM-UP TESTING



SYSTEM TESTING

- The system as a whole is tested to uncover requirement errors.
- Verifies that all system elements work properly and that overall system function and performance has been achieved.
- **Types:**
 - Alpha Testing
 - Beta Testing
 - Acceptance Testing
 - Performance Testing

Alpha Testing

It is carried out by the test team within the developing organization .

Beta Testing

It is performed by a selected group of friendly customers.

Acceptance Testing

It is performed by the customer to determine whether to accept or reject the delivery of the system.

Performance Testing

It is carried out to check whether the system meets the nonfunctional requirements identified in the SRS document.

TYPES OF PERFORMANCE TESTING

- Stress Testing
- Volume Testing
- Configuration Testing
- Compatibility Testing
- Regression Testing
- Recovery Testing
- Maintenance Testing
- Documentation Testing
- Usability Testing

Stress Testing -Stress testing is also known as *endurance testing*.

- Stress testing evaluates system performance when it is stressed for short periods of time.
- Stress tests are **black box tests** which are designed to impose a range of abnormal and even illegal input conditions so as to stress the capabilities of the software.
- Input data volume, input data rate, processing time, utilization of memory, etc. are tested beyond the designed capacity.
- For example, suppose an operating system is supposed to support 15 multi programmed jobs, the system is stressed by attempting to run 15 or more jobs simultaneously.
- A realtime system might be tested to determine the effect of simultaneous arrival of several high-priority interrupts.
- Stress testing is especially important for systems that usually operate below the maximum capacity but are severely stressed at some peak demand hours.
- For example, if the nonfunctional requirement specification states that the response time should not be more than 20 secs per transaction when 60 concurrent users are working, then during the stress testing the response time is checked with 60 users working simultaneously.

Volume Testing- It is especially important to check whether the data structures (arrays, queues, stacks, etc.) have been designed to successfully withstand extraordinary situations.

- For example, a compiler might be tested to check whether the symbol table overflows when a very large program is compiled.

Configuration Testing - This is used to analyse system behaviour in various hardware and software configurations specified in the requirements.

- Sometimes systems are built in variable configurations for different users. For instance, we might define a minimal system to serve a single user, and other extension configurations to serve additional users.
- The system is configured in each of the required configurations and it is checked if the system behaves correctly in all required configurations.

Compatibility Testing -This type of testing is required when the system interfaces with other types of systems.

- Compatibility aims to check whether the interface functions perform as required. For instance, if the system needs to communicate with a large database system to retrieve information, compatibility testing is required to test the speed and accuracy of data retrieval.

Regression Testing -This type of testing is required when the system being tested is an upgradation of an already existing system to fix some bugs or enhance functionality, performance, etc.

- Regression testing is the practice of running an old test suite after each change to the system or after each bug fix to ensure that no new bug has been introduced due to the change or the bug fix.
- If only a few statements are changed, then the entire test suite need not be run - only those test cases that test the functions that are likely to be affected by the change need to be run.

Recovery Testing -Recovery testing tests the response of the system to the presence of faults, or loss of power, devices, services, data, etc.

- The system is subjected to the loss of the mentioned resources (as applicable and discussed in the SRS document) and it is checked if the system recovers satisfactorily. For example, the printer can be disconnected to check if the system hangs. Or, the power may be shut down to check the extent of data loss and corruption.

Maintenance Testing- This testing addresses the diagnostic programs, and other procedures that are required to be developed to help maintenance of the system. It is verified that the artifacts exist and they perform properly.

Documentation Testing- It is checked that the required user manual, maintenance manuals, and technical manuals exist and are consistent. If the requirements specify the types of audience for which a specific manual should be designed, then the manual is checked for compliance.

Usability Testing- Usability testing concerns checking the user interface to see if it meets all user requirements concerning the user interface. During usability testing, the display screens, report formats, and other aspects relating to the user interface requirements are tested.

CONCLUSION

- In order to be cost effective, the testing must be concentrated on areas where it will be most effective.
- The testing should be planned such that when testing is stopped for whatever reason, the most effective testing in the time allotted has already been done.
- The absence of an organizational testing policy may result in too much effort and money will be spent on testing, attempting to achieve a level of quality that is impossible or unnecessary.